

GET / POST 요청 처리하기

□ 요청 방식이란?

○ HTTP에서 요청 방식이 중요한 이유

- HTTP 요청에는 다양한 방식(Method) 존재
- 서버는 요청의 목적에 따라 다르게 응답해야 하므로
요청 방식 자체가 서버 동작을 결정하는 중요한 신호

요청 방식	의미
GET	데이터 조회
POST	데이터 생성
PUT	데이터 전체 수정
PATCH	데이터 일부 수정
DELETE	데이터 삭제

○ GET과 POST 방식의 차이

구분	GET	POST
데이터 위치	URL 쿼리 파라미터	HTTP Body
목적	조회	전송/등록
보안성	낮음 (주소에 노출)	높음 (본문에 숨김)
데이터 길이	제한 있음 (주소 길이)	사실상 제한 없음
캐싱	브라우저 캐시됨	캐시되지 않음

※ URL 길이 제한 : HTTP 표준에는 길이 제한이 없지만 브라우저와 웹 서버에서 제한

브라우저	최대 길이 (URL 포함)
Chrome	약 32,768자 (32KB)
Firefox	약 65,536자 (64KB)
Safari	약 80,000자 이상
Internet Explorer	2,083자
Edge (Chromium)	약 32,768자
Opera	약 190,000자 이상

□ 요청 객체 (req)

속성	설명	예시값
req.method	요청 방식 (GET, POST 등)	'POST'
req.url	요청 경로 (쿼리 포함)	'/login?from=main'
req.headers	요청 헤더 정보 객체	{ 'content-type': 'application/json', ... }
req.query	쿼리스트링 파싱 결과 (GET용)	{ from: 'main' }
req.body	본문 데이터 (POST용, 미들웨어 필요)	{ username: 'nodejs' }
req.params	경로 파라미터 (/:id) 등	{ id: '123' }

□ 기본 http 모듈로 GET 처리하기

○ http 모듈의 기본 코드

```
const server = http.createServer((req, res) => {  
  });
```

○ req의 내부 모습 확인

```
console.log(req);
```

```
rawTrailers: [],  
joinDuplicateHeaders: null,  
aborted: false,  
upgrade: false,  
url: '/?name=abc&age=30&search=1234',  
method: 'GET',  
statusCode: null,  
statusMessage: null,
```

○ req.url을 url 모듈로 파싱

```
import url from 'node:url';  
const server = http.createServer((req, res) => {  
  console.log(req.url);  
  const parsedUrl = url.parse(req.url, true);  
  console.log(parsedUrl);  
});
```

```
Url {  
  protocol: null,  
  slashes: null,  
  auth: null,  
  host: null,  
  port: null,  
  hostname: null,  
  hash: null,  
  search: '?name=abc&age=30&search=1234',  
  query: [Object: null prototype] { name: 'abc', age: '30', search: '1234' },  
  pathname: '/',  
  path: '/?name=abc&age=30&search=1234',  
  href: '/?name=abc&age=30&search=1234'  
}
```

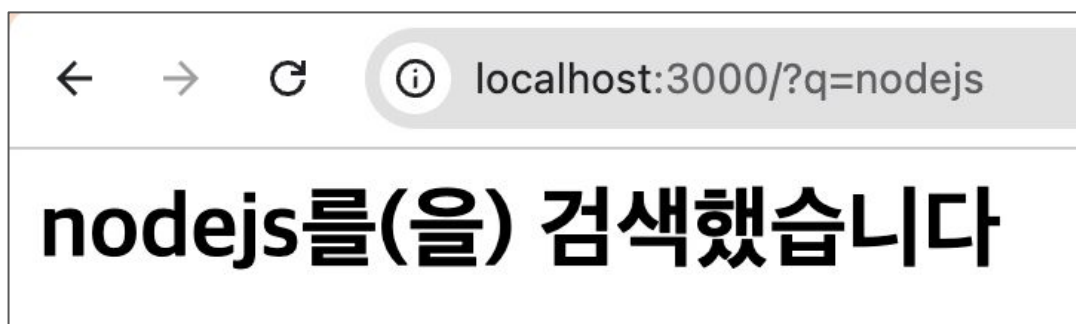
○ get-basic.js

```
import http from 'node:http';
import url from 'node:url';

const server = http.createServer((req, res) => {

  const parsedUrl = url.parse(req.url, true);
  const keyword = parsedUrl.query.q;
  res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
  res.end(`<h2>${keyword}를(을) 검색했습니다</h2>`);
});

server.listen(3000);
```



□ Express에서 GET 처리하기

○ Express의 기본 코드

```
app.get('/', (req, res) => {  
  });
```

○ req의 내부 모습 확인

```
console.log(req);
```

```
rawTrailers: [],  
joinDuplicateHeaders: null,  
aborted: false,  
upgrade: false,  
url: '/?name=abc&age=30&search=1234',  
method: 'GET',  
statusCode: null,  
statusMessage: null,
```

○ 파싱된 query 객체 제공

```
console.log(req.query);
```

```
[Object: null prototype] { name: 'abc', age: '30', search: '1234' }
```

○ get-express.js

```
import express from 'express';
```

```
const app = express();
```

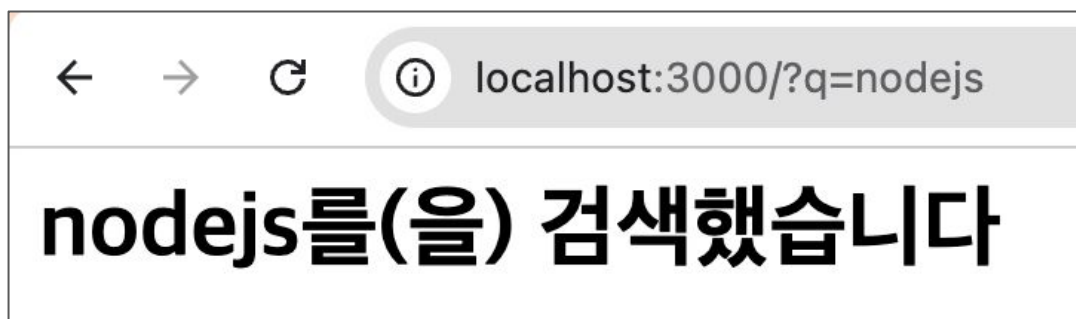
```
app.get('/', (req, res) => {
```

```
  const keyword = req.query.q;
```

```
  res.status(200).type('html').send(`<h2>${keyword}를(을) 검색했습니다</h2>`);
```

```
});
```

```
app.listen(3000);
```



□ 기본 http 모듈로 POST 처리하기

○ http 모듈의 기본 코드

```
const server = http.createServer((req, res) => {  
  if (req.method === 'POST') {  
    }  
});
```

○ POST는 데이터를 HTTP body에 담아 전송

- body 데이터가 chunk 단위로 나뉘어 들어오기 때문에 수동으로 조립

```
let body = "";  
req.on('data', chunk => {  
  body += chunk;  
});
```

- body 데이터: username=abcd&password=1234

- body의 문자열 데이터를 URLSearchParams 객체로 변환

```
const params = new URLSearchParams(body);  
URLSearchParams { 'username' => 'abcd', 'password' => '1234' }
```

○ post-basic.js

```
import http from 'node:http';
```

```
const server = http.createServer((req, res) => {  
  if (req.method === 'POST' && req.url === '/login') {  
    let body = "";  
    req.on('data', chunk => {  
      body += chunk;  
    });
```

```
    req.on('end', () => {  
      const params = new URLSearchParams(body);  
      const username = params.get('username');
```

```
      res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
```

```
      res.end(`<h2>${username} 님, 로그인 완료!</h2>`);
```

```
    });
```

```
  }
```

```
});
```

```
server.listen(3000);
```



□ Express에서 POST 처리하기

○ Express의 기본 코드

```
app.post('/login', (req, res) => {  
  });
```

○ 기본적으로 Express는 body 데이터에 접근할 수 없음

- **body-parser 역할**을 하는 **미들웨어를 등록**해야 **req.body** 사용 가능

```
app.use(express.urlencoded());
```

```
app.post('/login', (req, res) => {  
  console.log(req.body);  
  res.send("end");  
});
```

○ post-express.js

```
import express from 'express';
```

```
const app = express();
```

```
app.use(express.urlencoded());
```

```
app.post('/login', (req, res) => {  
  const { username } = req.body;  
  res.send(`<h2>${username} 님, 로그인 완료!</h2>`);  
});
```

```
app.listen(3000);
```



□ 기본 http 모듈로 JSON 요청 처리하기

○ http 모듈의 기본 코드

```
const server = http.createServer((req, res) => {  
  if (req.method === 'POST' && req.url === '/json') {  
    let body = "";  
  
    req.on('data', chunk => {  
      body += chunk; // { "name": "nodejs", "age": 20 }  
    });  
  }  
});
```

○ JSON 형태의 문자열을 객체 변환

```
const json = JSON.parse(body);
```

○ json-basic.js

```
import http from 'node:http';
```

```
const server = http.createServer((req, res) => {  
  if (req.method === 'POST' && req.url === '/json') {  
    let body = "";  
  
    req.on('data', chunk => {  
      body += chunk; // { "name": "nodejs", "age": 20 }  
    });  
  }  
});
```

```
req.on('end', () => {  
  const json = JSON.parse(body);
```

```
  const { name, age } = json;
```

```
  res.writeHead(200, { "Content-Type": "application/json; charset=utf-8" });
```

```
  res.end(JSON.stringify({ message: `name: ${name} / age: ${age}` }));
```

```
});
```

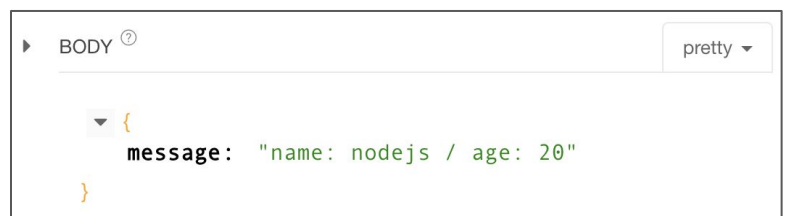
```
}
```

```
});
```

```
server.listen(3000, () => {
```

```
  console.log('http://localhost:3000/json 에 POST 요청 테스트 가능');
```

```
});
```



□ Express에서 JSON 요청 처리하기

○ Express의 기본 코드

```
app.post('/json', (req, res) => {  
  });
```

○ JSON 요청 데이터를 처리하기 위한 미들웨어 등록

- application/json일 때 req.body의 데이터를 객체로 변환

```
app.use(express.json());
```

○ json-express.js

```
import express from 'express';
```

```
const app = express();
```

```
app.use(express.json());
```

```
app.post('/json', (req, res) => {
```

```
  const { name, age } = req.body;
```

```
  res.json({ message: `name: ${name} / age: ${age}` });
```

```
});
```

```
app.listen(3000);
```

다양한 응답 방식과 상태 코드 처리

□ 응답(res)이란?

○ 개념 정의

- **res**는 서버가 클라이언트에게 응답을 보낼 때 사용하는 객체
- Node.js의 **http.createServer()**에서 두 번째 인자로 전달됨
- Express에서도 같은 이름(**res**)을 사용하며, 이를 확장하여 다양한 메서드를 제공
- 응답은 헤더(**header**)와 본문(**body**)로 구성됨
 - **header**: 브라우저가 응답을 어떻게 해석할지에 대한 정보
 - **body**: 실제 화면에 보여질 데이터

□ http 모듈에서 응답 처리

○ 주요 응답 메서드

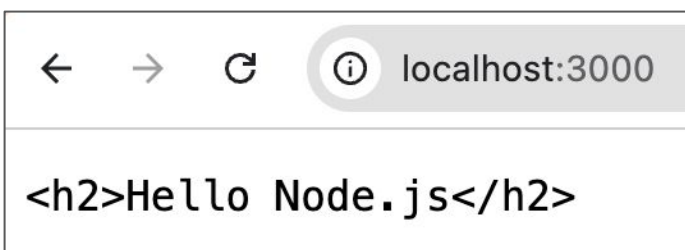
- **res.writeHead(statusCode, headers)**
 - 상태 코드와 응답 헤더 설정
- **res.write(data)**
 - 응답 본문 일부 작성
- **res.end([data])**
 - 응답 종료. **data**를 전달하면 **write** 없이 한 번에 작성 가능

○ 기본 http 모듈 - TEXT 응답 (res-basic-text.js)

```
import http from 'node:http';
```

```
const server = http.createServer((req, res) => {  
  res.write('<h2>Hello Node.js</h2>');  
  res.end();  
});
```

```
server.listen(3000);
```



□ 응답 상태 코드

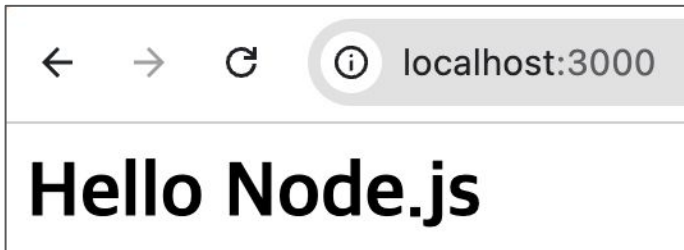
상태코드	의미	설명
200	OK	요청 성공
201	Created	자원 생성 완료
204	No Content	응답 본문 없음
301/302	Redirect	페이지 이동
400	Bad Request	잘못된 요청
401	Unauthorized	인증 필요
403	Forbidden	권한 없음
404	Not Found	경로 또는 자원 없음
500	Internal Server Error	서버 오류

- 기본 http 모듈 - HTML 응답 (res-basic-html.js)

```
import http from 'node:http';
```

```
const server = http.createServer((req, res) => {  
  res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });  
  res.write('<h2>Hello Node.js</h2>');  
  res.end();  
});
```

```
server.listen(3000);
```

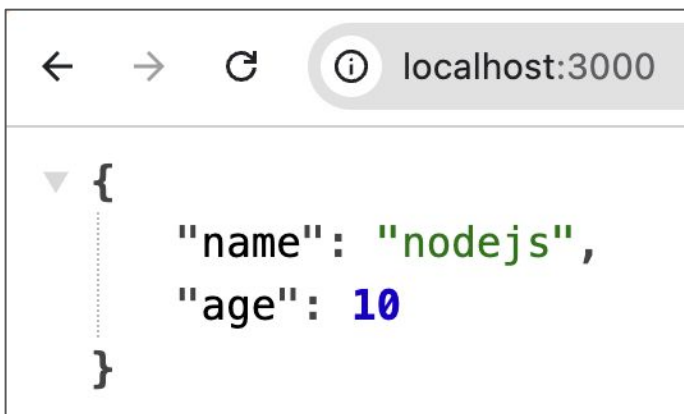


- 기본 http 모듈 - JSON 응답 (res-basic-json.js)

```
import http from 'node:http';
```

```
const server = http.createServer((req, res) => {  
  const data = { name: 'nodejs', age: 10 };  
  res.writeHead(200, { 'Content-Type': 'application/json; charset=utf-8' });  
  res.write(JSON.stringify(data));  
  res.end();  
});
```

```
server.listen(3000);
```



- 기본 http 모듈 - FILE 응답 (res-basic-file.js)

```
import http from 'node:http';
```

```
import fs from 'node:fs';
```

```
import path from 'node:path';
```

```
const filePath = path.join(process.cwd(), 'public', 'example.txt');
```

```
const stream = fs.createReadStream(filePath);
```

```
const server = http.createServer((req, res) => {
```

```
  res.writeHead(200, {
```

```
    'Content-Type': 'application/octet-stream',
```

```
    'Content-Disposition': 'attachment; filename="example.txt"'
```

```
  });
```

```
  stream.pipe(res);
```

```
});
```

```
server.listen(3000);
```

public/example.txt → 다운로드 연습용 파일

□ Express에서 응답 처리

○ Express의 주요 메서드

- `res.send(data)`
 - 텍스트, HTML, JSON 등 어떤 데이터든 자동으로 전송
- `res.json(obj)`
 - 객체를 JSON 형식으로 변환하여 전송
- `res.status(code)`
 - 상태 코드 설정 (단독 사용 또는 체이닝 가능)
- `res.redirect(url)`
 - 클라이언트를 다른 주소로 이동시킴

○ Express - TEXT 응답 (res-express-text.js)

```
import express from 'express';
```

```
const app = express();
```

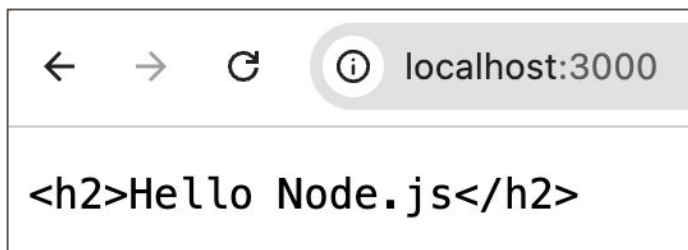
```
app.get('/', (req, res) => {
```

```
  res.set('Content-Type', 'text/plain; charset=utf-8'); // res.type('text');
```

```
  res.send('<h2>Hello Node.js</h2>');
```

```
});
```

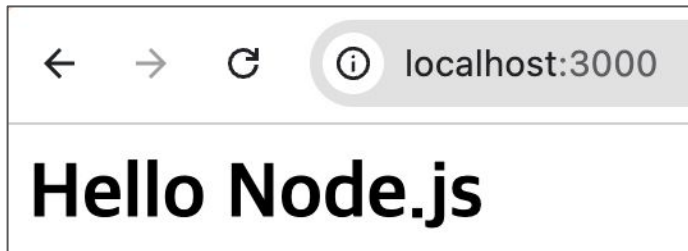
```
app.listen(3000);
```



- Express - HTML 응답 (res-express-html.js)

```
import express from 'express';  
const app = express();  
  
app.get('/', (req, res) => {  
  res.send('<h2>Hello Node.js</h2>');  
});
```

```
app.listen(3000);
```



- Express - JSON 응답 (res-express-json.js)

```
import express from 'express';  
const app = express();  
  
app.get('/', (req, res) => {  
  const data = { "name": "nodejs", "age": 10 };  
  res.send(data);  
});
```

```
app.listen(3000);
```

○ Express - FILE 응답 (res-express-file.js)

```
import express from 'express';
import path from 'node:path';

const app = express();

app.get('/', (req, res) => {
  const filePath = path.join(process.cwd(), 'public', 'example.txt');
  res.download(filePath); // 다운로드
  // res.sendFile(filePath); // 브라우저로 직접 열기
});

app.listen(3000);
```

public/example.txt → 다운로드 연습용 파일

○ Express - Redirect (res-express-redirect.js)

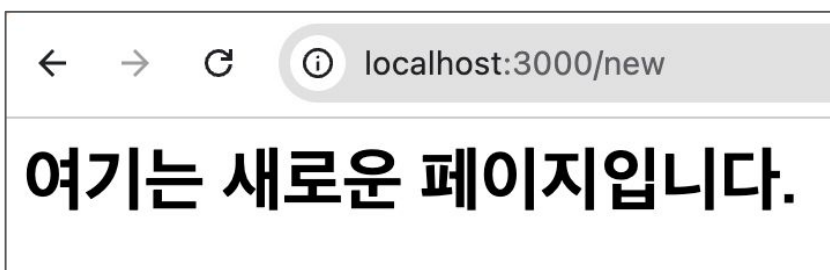
```
import express from 'express';

const app = express();

app.get('/old', (req, res) => {
  res.redirect('/new');
});

app.get('/new', (req, res) => {
  res.send('<h2>여기는 새로운 페이지입니다.</h2>');
});

app.listen(3000, () => {
  console.log('http://localhost:3000/old → /new로 리다이렉트됨');
});
```



[실습]

다운로드 방식 비교

1. 텍스트 파일 - public/sample.txt

"이 파일은 다운로드 테스트용 파일입니다."

2. 실행할 파일 - app-download-test.js

```
import express from 'express';
```

```
import path from 'node:path';
```

```
import fs from 'node:fs';
```

```
const app = express();
```

```
const __dirname = path.resolve();
```

```
const filePath = path.join(__dirname, 'public', 'sample.txt');
```

// 1. sendFile – 브라우저에서 직접 열기

```
app.get('/view', (req, res) => {
```

```
  res.sendFile(filePath);
```

```
});
```

// 2. download – 자동 다운로드

```
app.get('/download', (req, res) => {
```

```
  res.download(filePath, '자동다운로드.txt');
```

```
});
```

// 3. 수동 헤더 설정 – 완전 수동 다운로드

```
app.get('/manual', (req, res) => {
```

```
  res.writeHead(200, {
```

```
    'Content-Type': 'application/octet-stream',
```

```
    'Content-Disposition': `attachment; filename="${encodeURIComponent('수동다운로드.txt')}"`,
```

```
  });
```

```
const stream = fs.createReadStream(filePath);
```

```
stream.pipe(res);
```

```
});
```

```
app.listen(3000, () => {
```

```
  console.log('http://localhost:3000/view    → 브라우저에서 보기');
```

```
  console.log('http://localhost:3000/download → 자동 다운로드');
```

```
  console.log('http://localhost:3000/manual   → 수동 다운로드');
```

```
});
```

3. 동작 확인

```
node app-download-test.js
```

기능	URL	기대 동작
파일 열기	http://localhost:3000/view	브라우저에서 바로 보기
자동 다운로드	http://localhost:3000/download	파일 저장 창 열림
수동 다운로드	http://localhost:3000/manual	파일 저장 창 열림

※ 정리

방식	파일 이름 지정	헤더 설정 필요	결과
sendFile	×	×	브라우저에서 열기
download	○	×	자동 다운로드 (내부적으로 헤더 설정)
수동 구현	○	○	완전 수동 다운로드

연습문제

1. 이미지 직접 보기 / 다운로드 기능 구현 (image-header.js)

- 설명

· 요청에 따라 이미지를 브라우저로 바로 보여주거나 파일로 다운로드되도록 응답하는 서버를 작성합니다.

- 요구사항

- [public/nodejs.png](#) 이미지를 활용
- 요청 경로 `/image`는 이미지를 브라우저에서 직접 보기
 - `res.sendFile()`만 사용 (추가 헤더 설정 없음)
- 요청 경로 `/download`는 이미지를 다운로드
 - `res.sendFile()`을 사용하되 `Content-Disposition` 헤더를 추가로 설정해서 다운로드되도록 만들기

- 출력 예시

요청: <http://localhost:3000/image>

→ 브라우저에 `nodejs.png` 이미지 표시

요청: <http://localhost:3000/download>

```
import express from 'express'; 이미지.png로 저장
import path from 'node:path';
```

```
const app = express();
```

```
// 코드 작성 1) nodejs.png 파일의 절대 경로 만들기
```

```
// path.resolve('폴더명', '파일명'): 현재 실행 경로 기준으로 절대 경로 생성
```

```
// 코드 작성 2) '/image' 경로 - 브라우저에서 바로 이미지 보여주기
```

```
// res.sendFile(파일경로) 만 호출하기 (추가 헤더 설정 없음)
```

```
// 코드 작성 3) '/download' 경로 - 이미지를 다운로드시키기
```

```
// res.set('Content-Disposition', 'attachment; filename="이미지.png") 로 헤더 먼저 설정
```

```
// 그 다음 res.sendFile(파일경로) 호출
```

```
app.listen(3000, () => {
```

```
  console.log('http://localhost:3000/image → 브라우저에서 보기');
```

```
  console.log('http://localhost:3000/download → 다운로드');
```

```
});
```